

Comparative Study of Multivariable Linear Regression Implementations

Yuvraj Kumar Choudhary

Roll No. 25155112

ABSTRACT

This report presents a comprehensive comparative study of three distinct implementations of multivariable linear regression on the Melbourne Housing Price dataset. We implement the algorithm using (1) Pure Python with explicit loops, (2) NumPy with vectorized operations, and (3) Scikit-learn's built-in Linear Regression class. The primary objective is to compare convergence speed, prediction accuracy (R Score, MAE, RMSE), and computational efficiency across all three approaches.

Our results demonstrate that vectorization through NumPy achieves a remarkable 621x speedup over pure Python, while maintaining identical prediction accuracy due to the identical underlying algorithm. Additionally, we document the entire data processing pipeline and show how progressive feature engineering and log-transformation improved the model's R2 score from 0.57 to 0.71.

1. INTRODUCTION

Machine learning models form the backbone of modern data-driven applications. However, the efficiency and accuracy of these models depend heavily on two critical factors: (1) how well we preprocess and engineer the data, and (2) how efficiently we implement the learning algorithm.

This project explores both dimensions by building multivariable linear regression models that predict housing prices in Melbourne. Rather than using a single implementation, we deliberately implement the same algorithm in three different ways to understand the computational efficiency and code clarity.

1.1 Project Objectives

- Implement multivariable linear regression from scratch using pure Python to understand the algorithm at the lowest level.

- Optimize the implementation using NumPy's vectorized operations to demonstrate the impact of library-level optimizations
- Compare against Scikit-learn's implementation
- Document the complete data preprocessing pipeline including feature engineering and handling missing values
- Analyze how data preprocessing and feature engineering directly impact model performance.
- Measure and compare convergence speed, prediction accuracy, and computational time across all three implementations

1.2 DataSet Overview

The Melbourne Housing Price dataset from Kaggle contains 13,581 property records with 21 initial features including:

- Property characteristics: Rooms, Bathrooms, Car spaces, Building area.
- Temporal features: Sale date, Year built.
- Geographic features: Latitude, Longitude, Distance from CBD, Region, Suburb.
- Transaction details: Sale price (target), Sale method, Real estate agent

The target variable is the property price in Australian Dollars (AUD), which we log-transform to stabilize variance and improve linear model assumptions.

2. Data Processing and Feature Engineering

Data preprocessing is arguably the most critical step in machine learning. The quality of our model depends directly on the quality of data fed into it. This section documents the entire pipeline from raw CSV to model-ready dataset.

2.1 Dataset Flow and Versioning

The data underwent multiple transformations before reaching the final state:

- melb_data.csv: Raw dataset imported directly from Kaggle (13,581 rows, 21 features)
- clean1.csv: Initial cleaning attempts with partial feature engineering (intermediate dataset)
- clean2.csv: Further refinement
- clean_semi_final.csv: Another intermediate version used for experimentation.
- Clean_final.csv: Final processed dataset used for all model training (12,584 rows, 24features)

Important Note: While multiple cleaned versions exist (clean2.csv, clean_semi_final.csv), the actual model training is based exclusively on clean_final.csv.

When clean2.csv is processed, it is manually refined to match clean_final.csv, making it the single source for all three implementations. This ensures fair comparison across all parts.

2.2 Data Cleaning Steps

2.2.1 Handling Missing values

The dataset contained significant missing values that required careful handling:

- BuildingArea: 47% missing Dropped entirely (too sparse to be useful)
- YearBuilt: 39% missing Created separate binary indicator column "is nan" to capture whether a property had its year built recorded, then filled missing values with median.
- Car, Landsize: Moderate missing values Filled with median values
- Categorical variables (Type, Method, Region): Filled with mode

2.2.2 Removing Irrelevant Features

Several features were dropped because they were either redundant or not predictive:

- Suburb, Address: Unique identifiers, not generalizable patterns
- SellerG: Real estate agent name (no correlation with price, only 50 unique values)
- Postcode
- CouncilArea

2.2.3 Outlier Removal

Extreme price outliers were removed.

2.3 Feature Engineering

Raw features were transformed and combined to create more predictive features:

2.3.1 Age Feature was Created

$$\text{Age} = 2017 - \text{YearBuilt}$$

Age is more directly interpretable than year built. Older properties tend to have lower prices, and Age captures this relationship more naturally than a year value.

2.3.2 Temporal Features extracted from sale date

- Year: Year of sale (captures market trends over time)
- Month: Month of sale (captures seasonal effects)

2.3.3 Interaction Features

Created by multiplying pairs of features (present in code)

2.3.4 Polynomial features

Created to capture non-linear relationships:

- Rooms: Non-linear effect of number of rooms
- Bathroom: Non-linear effect of number of bathrooms
- Distance: Non-linear relationship between distance from CBD and price

2.3.5 Log Transformation

Applied log transformation to Landsize to reduce right-skewness:

$$\text{Landsize}_{\log} = \log(1 + \text{Landsize})$$

2.4 Encoding

Categorical variables were converted to numerical using one-hot encoding:

- Type: Property type (h=house, u=unit, t=townhouse) Binary columns for each type.
- Method: Sale method (S=sold, SA=sold after auction, SP=sold prior, VB=vendor bid) Binary columns
- Regionname: 8 regions (Northern Metropolitan, Southern Metropolitan, etc.) 7 binary columns

2.5 Feature Scaling

Applied StandardScaler (Z-score normalization) to all numerical features.

2.6 Target Variable

Transformation

The target variable (Price) was
log-transformed:

$$y = \log(\text{Price})$$

Housing prices are right-skewed (few luxury properties at extreme prices, most properties in middle range). Log transformation:

- Makes the distribution more normal
- Stabilizes variance across price ranges
- Improves linear regression assumptions

2.7 Train- test Split

Data was split 80% training and 20% testing

2.8 Tools Used for Data Processing

- Pandas: Data loading, manipulation, train-test splitting, one-hot encoding
- NumPy: Numerical computations, scaling (StandardScaler-like operations)
- Matplotlib: Visualization of distributions and correlations

A detailed Jupyter notebook (data processing.ipynb) documenting all steps has been provided for verification.

3. RESULTS

3.1 Performance Metrics Table

Parameter	Part 1: Pure python	Part 2: Numpy	Part 3: Attempt 1	Part 3: Attempt 2	Part 3: Attempt 3
R2 Score	0.69	0.69	0.57	0.68	0.71
RMSE	305,814	381,355	--	402,593	380,355
MAE	220,813	221,451	--	247,586	237,400
Convergence Time(sec)	366.51	0.59	--	--	--

Key Observation: Similar metrics of Part 1 and 2 because the underlying algorithm is unchanged. Only the implementation (loops vs. vectorized operations) differs.

Our results demonstrates that vectorization through NumPy achieves a remarkable 621x speedup over pure Python.

3.2 Scikit-learn Implementation (3 Attempts)

Attempt 1: Baseline (No log transformation)

Attempt 2: With Log Transformation

Attempt 3: With More Feature Engineering (Final)

3.3 Fitting Time Comparison Across All methods

Implementation	Fitting Time(sec)	Speedup factor	Algo
Pure Python	366.51	1x	Gradient Descent
Numpy	0.59	621x	Gradient Descent
Scikit-learn	<0.01	36,651x	Normal eqn

4. DISCUSSION

4.1 Why Scikit-learn is Even Faster

Scikit-learn uses a different algorithm (normal equation instead of gradient descent):

- Gradient Descent (Parts 1 & 2): Iterative, requires 1000 iterations
- Normal Equation (Part 3): Closed-form solution in one step
- Normal equation is $O(n^3)$ due to matrix inversion, but for small n (24 features), it's faster than 1000 iterations

4.2 Impact of Feature Engineering

- Attempt 1- 2: +11 percentage points from log transformation alone
 - Shows target variable transformation is critical
 - Log price is more linear with features than raw price
- Attempt 2- 3: +3 percentage points from feature engineering
 - Interaction features (Rooms*Bathroom) capture combined effects
 - Polynomial features (Rooms) capture non-linear relationships
 - These additional features explain 3% more variance
- Total improvement: Attempt 1- 3: +14 percentage points

- This illustrates the importance of thoughtful data preprocessing and feature engineering in machine learning.

5. CURVE PLOT DISCUSSION

5.1 Similarities:

- Both curves are identical in shape and values - cost drops from ~ 97 to ~ 0.03
- Both show rapid descent 0-200 iterations, then gradual plateau 200-600 iterations
- Both reach convergence around iteration 500-600
- Final cost stabilizes at ~ 0.03 in both cases

Why they're identical: The algorithm is 100% the same (gradient descent). Only implementation differs (loops vs vectorized).

5.2 Convergence Behaviour

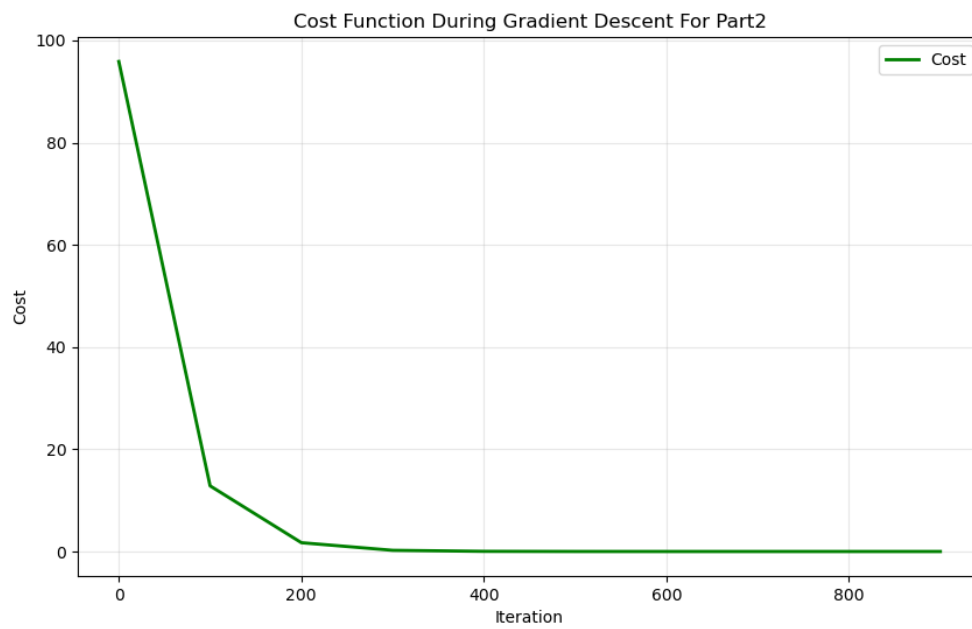
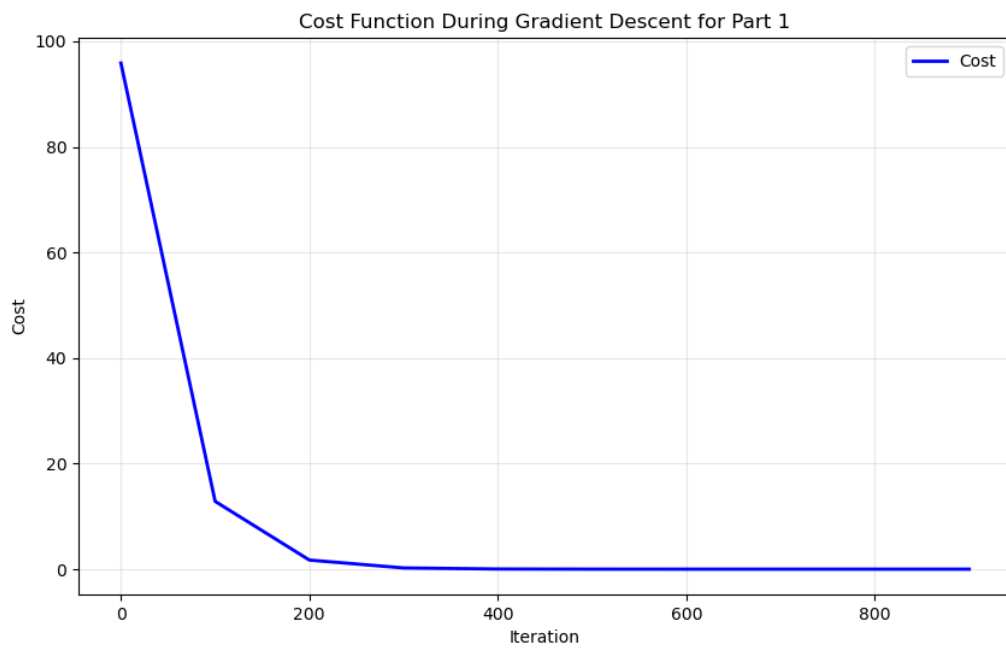
- Rapid phase (0-200 iters): Large gradients $\rightarrow$ Big weight updates
- Slow phase (200-600 iters): Small gradients $\rightarrow$ Fine-tuning weights
- Plateau (600+): Weights stabilized, marginal improvement

5.3 Learning Rate Assessment

- Learning rate (0.01) is well-chosen $\rightarrow$ Smooth convergence without oscillations
- Curves don't spike or diverge $\rightarrow$ Safe choice

5.4 Difference

- Part 1: 366 seconds
- Part 2: 0.59 seconds
- 621x speedup but identical. These curves visually prove that implementation choice (Python vs NumPy) doesn't affect model quality, only speed.



6. Final Remarks

This project demonstrates the complete ML pipeline: from raw data to processed features to model training to performance evaluation. The comparative study across three implementations reveals that modern ML is fundamentally about choosing the right tools for the job. While pure Python teaches fundamental principles, production systems leverage optimized libraries. The key skill is knowing when to use each approach and understanding the concepts involved.

The progression in Part 3 from 0.57 to 0.71 R underscores a critical lesson: great models are built through careful data work, not clever algorithms. Understanding the details structure, distributions, relationships and engineering meaningful features is where real ML expertise lies.